МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ АВТОМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ
Кафедра автоматизированных систем управления

ОТЧЕТ

по дисциплине Современные технологии разработки программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА № 6

Тема: Библиотека Android Design Support Library

Выполнил:
Епифанцев Г.А.
Группа АПМ-25

Подпись: _____ Дата: ___

Проверил:
Эстрайх И.В.

Оценка _____

Подпись: _____ Дата: ___

Новосибирск 2026

СОДЕРЖАНИЕ

Постановка задачи **3**

Введение **4**

1 Ход работы **5**

1.1 Структура экрана Material Design 5

1.2 Базовый Snackbar 6

1.3 Продолжительность и ручное закрытие 7

1.4 События Snackbar 8

1.5 Изменение цветов 9

1.6 Кнопка действия 10

1.7 Смещение нижних компонентов 11

1.8 Закруглённый Snackbar 11

Заключение **12**

Приложение А. Исходный код приложений **13**

ПОСТАНОВКА ЗАДАЧИ

Цель работы

Освоить компоненты Material Design, предназначенные для улучшения внешнего вида и поведения Android-приложений.

Задание

1. Создать приложения на Java и Kotlin, использующие возможности библиотеки Android Design Support, рассмотренные в методических указаниях.
2. Включить в отчёт цель работы, код программ, скриншоты результатов и вывод.

ВВЕДЕНИЕ

Библиотека Android Design Support предоставляла готовые компоненты Material Design для старых версий Android. В актуальном проекте ее возможности доступны через AndroidX и Material Components. Названия основных компонентов и принцип их совместной работы сохранились, но пакеты android.support.* заменены на androidx.* и com.google.android.material.*.

Для выполнения работы созданы два самостоятельных проекта Android Studio. Lab6MaterialJava содержит реализацию на Java, а Lab6MaterialKotlin — реализацию на Kotlin. Оба приложения можно установить на одно устройство, поскольку у них разные идентификаторы приложения.

1 Ход работы

1.1 Структура экрана Material Design

Корневым элементом activity_main.xml является CoordinatorLayout. В нём размещены AppBarLayout с вложенным Toolbar, подключаемая разметка content_main.xml, нижняя информационная панель и FloatingActionButton. Такая структура позволяет Snackbar координировать свое положение с FAB и нижней панелью.

```
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_margin="@dimen/fab_margin"
    android:contentDescription="@string/fab_description"
    app:backgroundTint="@color/amber_accent"
    app:srcCompat="@drawable/ic_swift"
    app:tint="@null" />
```

Для FAB использован векторный значок Swift. Основные элементы интерфейса зелёные, а FAB и кнопки действия жёлтые.

Основные экраны Java- и Kotlin-приложений показаны на рисунках 1 и 2. Структура и набор действий в обоих приложениях одинаковые.

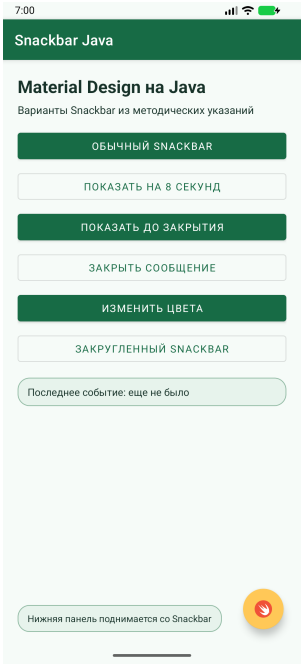


Рисунок 1 — Основной экран Java-приложения

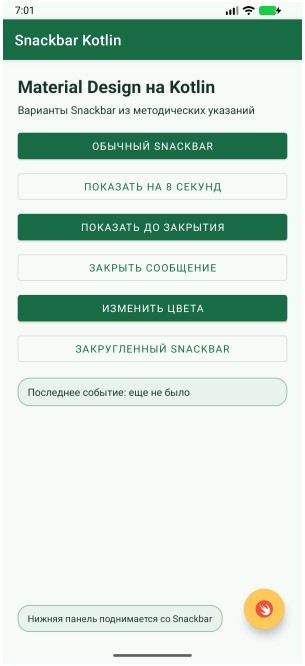


Рисунок 2 — Основной экран Kotlin-приложения

1.2 Базовый Snackbar

В Java-приложении объект Snackbar создаётся методом make(). Первым параметром передаётся нажатая кнопка. По ней библиотека находит родительский CoordinatorLayout. Второй параметр задаёт текст, третий — продолжительность показа.

```
findViewById(R.id.basicButton).setOnClickListener(view ->
        showTracked(Snackbar.make(
                view,
                R.string.basic_message,
                Snackbar.LENGTH_LONG
        ))
);
```

Kotlin-вариант использует тот же класс и лямбда-выражение обработчика.

```
findViewById<View>(R.id.basicButton).setOnClickListener { view ->
    showTracked(
        Snackbar.make(view, R.string.basic_message, Snackbar.LENGTH_LONG),
    )
}
```

На рисунке 3 видно, как после нажатия кнопки Snackbar появился у нижней границы приложения, а FAB и нижняя информационная панель поднялись над сообщением.

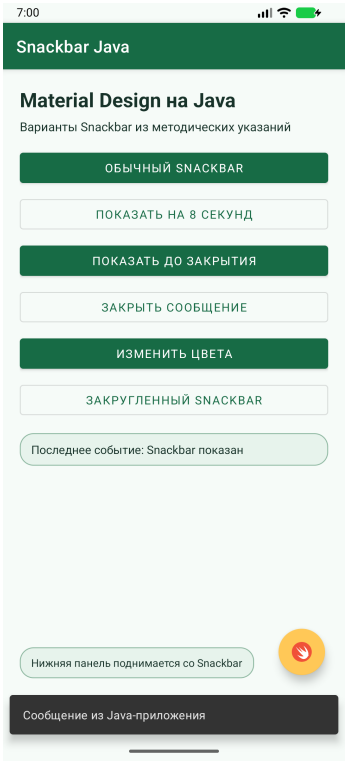


Рисунок 3 — Базовый Snackbar и поднятие нижних компонентов

1.3 Продолжительность и ручное закрытие

Для проверки произвольной продолжительности добавлена отдельная кнопка. Значение 8000 задаёт время показа в миллисекундах.

```
findViewById<View>(R.id.durationButton).setOnClickListener { view ->
    showTracked(Snackbar.make(view, R.string.duration_message, 8000))
}
```

Вариант LENGTH_INDEFINITE не исчезает по тайм-ауту. Ссылка на активную панель хранится в поле activeSnackbar, поэтому другая кнопка может закрыть ее методом dismiss().

```
findViewById<View>(R.id.indefiniteButton).setOnClickListener { view ->
    showTracked(
        Snackbar.make(view, R.string.indefinite_message, Snackbar.
    LENGTH_INDEFINITE),
    )
}

findViewById<View>(R.id.dismissButton).setOnClickListener {
    activeSnackbar?.dismiss()
}
```

На рисунке 4 показан Snackbar, который закрывается через 8 секунд. Вариант с неопределённой продолжительностью показан на рисунке 5.

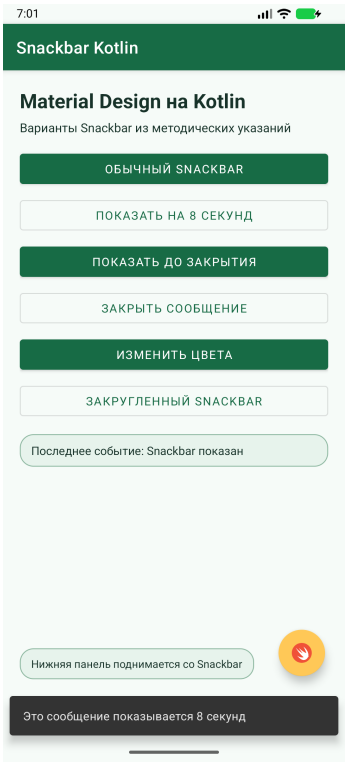


Рисунок 4 — Snackbar с показом в течение 8 секунд

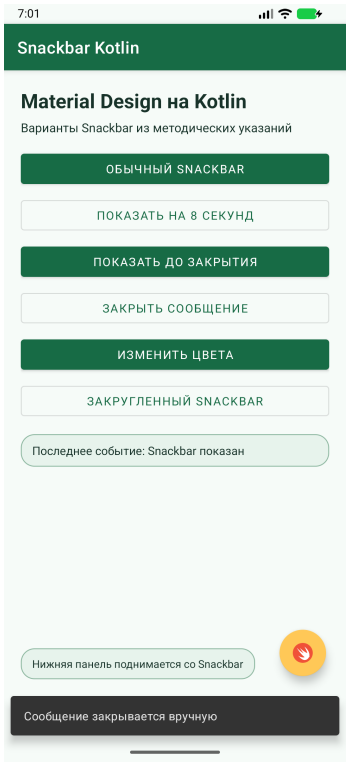


Рисунок 5 — Snackbar до ручного закрытия

1.4 События Snackbar

Метод addCallback() позволяет определить момент появления панели и причину ее закрытия. Событие записывается в Logcat и выводится в поле eventText.

```
snackbar.addCallback(new Snackbar.Callback() {
    @Override
    public void onShown(Snackbar shownSnackbar) {
        updateEvent("Snackbar показан");
    }

    @Override
    public void onDismissed(Snackbar dismissedSnackbar, int event) {
        updateEvent(dismissReason(event));
        if (activeSnackbar == dismissedSnackbar) {
            activeSnackbar = null;
        }
    }
});
```

На рисунке 6 видно, как после вызова dismiss() в поле состояния появился текст «Последнее событие: закрыт вручную».

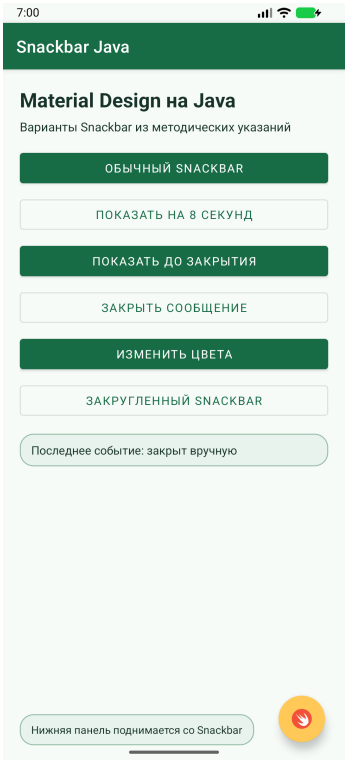


Рисунок 6 — Результат обработки события ручного закрытия

1.5 Изменение цветов

Фон панели задаёт метод setBackgroundTint(), цвет сообщения — setTextColor(). Цвет кнопки действия меняется с помощью setActionTextColor().

```
Snackbar snackbar = Snackbar.make(
        view,
        R.string.colored_message,
        Snackbar.LENGTH_INDEFINITE
)
        .setBackgroundTint(getColor(R.color.green_primary))
        .setTextColor(getColor(R.color.snackbar_text))
        .setActionTextColor(getColor(R.color.amber_accent))
        .setAction(R.string.dismiss_action, button ->
                updateEvent("нажата кнопка закрытия")
        );
```

На рисунке 7 видны зелёный фон, светлый текст и жёлтая кнопка «Закрыть».

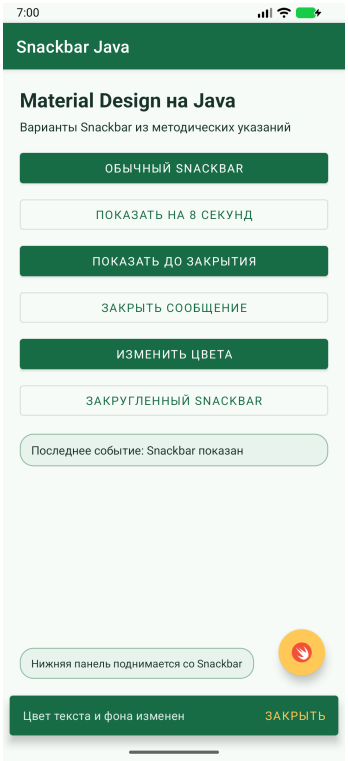


Рисунок 7 — Изменение фона, текста и кнопки действия

1.6 Кнопка действия

Метод setAction() получает текст кнопки и обработчик нажатия. В этой работе FAB открывает панель с вопросом «Изучить Swift?». При нажатии «Да» Snackbar закрывается и появляется Toast «Отлично!».

```
val snackbar = Snackbar.make(
    view,
    R.string.action_message,
    Snackbar.LENGTH_INDEFINITE,
)
    .setBackgroundTint(getColor(R.color.green_dark))
    .setTextColor(getColor(R.color.snackbar_text))
    .setActionTextColor(getColor(R.color.amber_accent))
    .setAction(R.string.action_yes) {
        Toast.makeText(
            applicationContext,
            R.string.action_result,
            Toast.LENGTH_SHORT,
        ).show()
    }
```

Панель до нажатия «Да» показана на рисунке 8. После выполнения действия появляется Toast, показанный на рисунке 9.

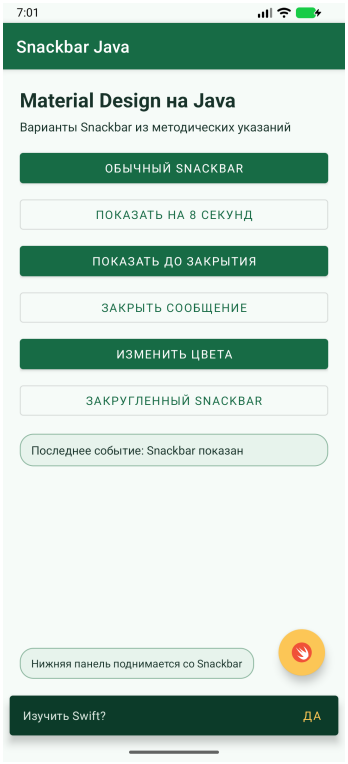


Рисунок 8 — Snackbar с кнопкой «Да»

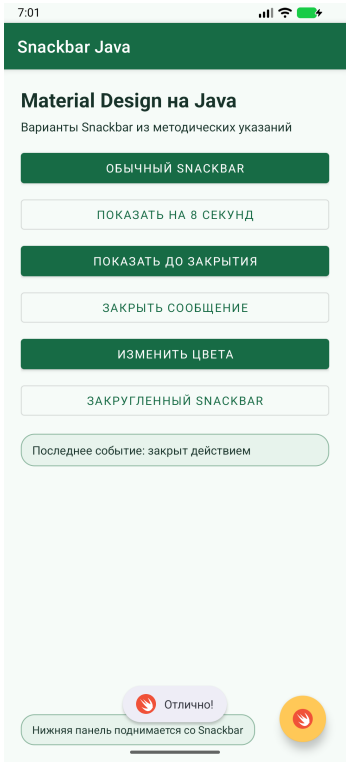


Рисунок 9 — Toast после нажатия кнопки

1.7 Смещение нижних компонентов

Нижняя информационная панель является прямым дочерним элементом CoordinatorLayout. Атрибут layout_dodgeInsetEdges заставляет ее перемещаться выше Snackbar.

```
<TextView
    android:id="@+id/bottomPanel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|start"
    app:layout_dodgeInsetEdges="bottom" />
```

Результат виден на рисунках 3, 5, 7 и 8. Нижняя панель и FAB остаются над Snackbar.

1.8 Закруглённый Snackbar

Фон панели задан в ресурсе bg_snackbar_rounded.xml. Радиус скругления углов составляет 18 dp.

Вспомогательный класс получает представление Snackbar, задаёт внешние отступы и фон, а также высоту подъёма панели для отображения тени. Отступы задаются в dp и переводятся в пиксели через ресурсы приложения.

На рисунке 10 видны внешние отступы и закруглённые углы панели.

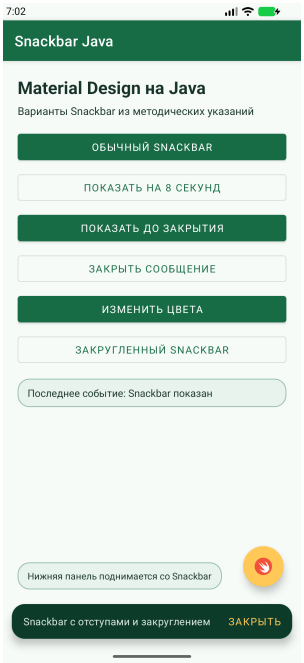


Рисунок 10 — Snackbar с отступами и закруглённым фоном

ЗАКЛЮЧЕНИЕ

В ходе работы созданы два Android-приложения на Java и Kotlin. Для разметки использованы CoordinatorLayout, AppBarLayout и Toolbar. Взаимодействие с пользователем реализовано с помощью FloatingActionButton и Snackbar. Проверены стандартная и заданная продолжительность показа, ручное закрытие и обработка событий. Изменены цвета панели и добавлена кнопка действия. Реализован закруглённый фон с отступами, а нижняя панель и FAB поднимаются при появлении Snackbar. Устаревшие пакеты Android Design Support заменены на AndroidX и Material Components. Оба приложения запущены и проверены на эмуляторе Android с уровнем API 37.

ПРИЛОЖЕНИЕ А

Исходный код приложений

Java. MainActivity.java

```
package ru.nstu.lab6.javaapp;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;

import com.google.android.material.floatingactionbutton.
   FloatingActionButton;
import com.google.android.material.snackbar.Snackbar;

public class MainActivity extends AppCompatActivity {
    private static final String TAG = "Lab6Snackbar";
    private Snackbar activeSnackbar;
    private TextView eventText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);

        eventText = findViewById(R.id.eventText);

        findViewById(R.id.basicButton).setOnClickListener(view ->
                showTracked(Snackbar.make(view, R.string.basic_message,
   Snackbar.LENGTH_LONG))
        );

        findViewById(R.id.durationButton).setOnClickListener(view ->
                showTracked(Snackbar.make(view, R.string.duration_message,
   8000))
        );

        findViewById(R.id.indefiniteButton).setOnClickListener(view ->
                showTracked(Snackbar.make(view, R.string.indefinite_message
   , Snackbar.LENGTH_INDEFINITE))
        );

        findViewById(R.id.dismissButton).setOnClickListener(view -> {
            if (activeSnackbar != null) {
                activeSnackbar.dismiss();
            }
```

```
        });

        findViewById(R.id.colorButton).setOnClickListener(this::
showColoredSnackbar);
        findViewById(R.id.roundedButton).setOnClickListener(this::
showRoundedSnackbar);

        FloatingActionButton fab = findViewById(R.id.fab);
        fab.setOnClickListener(this::showActionSnackbar);
    }

    private void showColoredSnackbar(View view) {
        Snackbar snackbar = Snackbar.make(view, R.string.colored_message,
Snackbar.LENGTH_INDEFINITE)
                .setBackgroundTint(getColor(R.color.green_primary))
                .setTextColor(getColor(R.color.snackbar_text))
                .setActionTextColor(getColor(R.color.amber_accent))
                .setAction(R.string.dismiss_action, button -> updateEvent("
нажата кнопка закрытия"));
        showTracked(snackbar);
    }

    private void showActionSnackbar(View view) {
        Snackbar snackbar = Snackbar.make(view, R.string.action_message,
Snackbar.LENGTH_INDEFINITE)
                .setBackgroundTint(getColor(R.color.green_dark))
                .setTextColor(getColor(R.color.snackbar_text))
                .setActionTextColor(getColor(R.color.amber_accent))
                .setAction(R.string.action_yes, button ->
                        Toast.makeText(getApplicationContext(), R.string.
action_result, Toast.LENGTH_SHORT).show()
                );
        showTracked(snackbar);
    }

    private void showRoundedSnackbar(View view) {
        Snackbar snackbar = Snackbar.make(view, R.string.rounded_message,
Snackbar.LENGTH_INDEFINITE)
                .setAction(R.string.dismiss_action, button -> updateEvent("
закругленная панель закрыта"));
        SnackbarHelper.configure(this, snackbar);
        showTracked(snackbar);
    }

    private void showTracked(Snackbar snackbar) {
        activeSnackbar = snackbar;
        snackbar.addCallback(new Snackbar.Callback() {
            @Override
            public void onShown(Snackbar shownSnackbar) {
                updateEvent("Snackbar показан");
            }

            @Override
            public void onDismissed(Snackbar dismissedSnackbar, int event)
{
                updateEvent(dismissReason(event));
                if (activeSnackbar == dismissedSnackbar) {
                    activeSnackbar = null;
                }
            }
```

```
        });
        snackbar.show();
    }

    private String dismissReason(int event) {
        switch (event) {
            case Snackbar.Callback.DISMISS_EVENT_SWIPE:
                return "закрыт смахиванием";
            case Snackbar.Callback.DISMISS_EVENT_ACTION:
                return "закрыт действием";
            case Snackbar.Callback.DISMISS_EVENT_TIMEOUT:
                return "закрыт по таймауту-";
            case Snackbar.Callback.DISMISS_EVENT_MANUAL:
                return "закрыт вручную";
            case Snackbar.Callback.DISMISS_EVENT_CONSECUTIVE:
                return "заменен новым Snackbar";
            default:
                return "закрыт";
        }
    }

    private void updateEvent(String event) {
        Log.i(TAG, event);
        eventText.setText(getString(R.string.event_format, event));
    }
}
```

Java. SnackbarHelper.java

```
package ru.nstu.lab6.javaapp;

import android.content.Context;
import android.view.View;
import android.view.ViewGroup;

import androidx.core.view.ViewCompat;

import com.google.android.material.snackbar.Snackbar;

final class SnackbarHelper {
    private SnackbarHelper() {
    }

    static void configure(Context context, Snackbar snackbar) {
        View snackbarView = snackbar.getView();
        ViewGroup.LayoutParams layoutParams = snackbarView.getLayoutParams
   ();
        if (layoutParams instanceof ViewGroup.MarginLayoutParams) {
            ViewGroup.MarginLayoutParams marginParams =
                    (ViewGroup.MarginLayoutParams) layoutParams;
            int margin = context.getResources().getDimensionPixelSize(R.
   dimen.snackbar_margin);
            marginParams.setMargins(margin, margin, margin, margin);
            snackbarView.setLayoutParams(marginParams);
        }
        snackbarView.setBackgroundResource(R.drawable.bg_snackbar_rounded);
        ViewCompat.setElevation(
                snackbarView,
                context.getResources().getDimension(R.dimen.
   snackbar_elevation)
        );
        snackbar.setTextColor(context.getColor(R.color.snackbar_text));
        snackbar.setActionTextColor(context.getColor(R.color.amber_accent))
   ;
    }
}
```

Kotlin. MainActivity.kt

```
package ru.nstu.lab6.kotlinapp

import android.os.Bundle
import android.util.Log
import android.view.View
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.Toolbar
import com.google.android.material.floatingactionbutton.
   FloatingActionButton
import com.google.android.material.snackbar.Snackbar

class MainActivity : AppCompatActivity() {
    private var activeSnackbar: Snackbar? = null
    private lateinit var eventText: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar: Toolbar = findViewById(R.id.toolbar)
        setSupportActionBar(toolbar)

        eventText = findViewById(R.id.eventText)

        findViewById<View>(R.id.basicButton).setOnClickListener { view ->
            showTracked(Snackbar.make(view, R.string.basic_message,
   Snackbar.LENGTH_LONG))
        }

        findViewById<View>(R.id.durationButton).setOnClickListener { view
   ->
            showTracked(Snackbar.make(view, R.string.duration_message,
   8000))
        }

        findViewById<View>(R.id.indefiniteButton).setOnClickListener { view
    ->
            showTracked(Snackbar.make(view, R.string.indefinite_message,
   Snackbar.LENGTH_INDEFINITE))
        }

        findViewById<View>(R.id.dismissButton).setOnClickListener {
            activeSnackbar?.dismiss()
        }

        findViewById<View>(R.id.colorButton).setOnClickListener(::
   showColoredSnackbar)
        findViewById<View>(R.id.roundedButton).setOnClickListener(::
   showRoundedSnackbar)

        findViewById<FloatingActionButton>(R.id.fab).setOnClickListener(::
   showActionSnackbar)
    }

    private fun showColoredSnackbar(view: View) {
```

```
    val snackbar = Snackbar.make(view, R.string.colored_message,
Snackbar.LENGTH_INDEFINITE)
        .setBackgroundTint(getColor(R.color.green_primary))
        .setTextColor(getColor(R.color.snackbar_text))
        .setActionTextColor(getColor(R.color.amber_accent))
        .setAction(R.string.dismiss_action) {
            updateEvent("нажата кнопка закрытия")
        }
    showTracked(snackbar)
}

private fun showActionSnackbar(view: View) {
    val snackbar = Snackbar.make(view, R.string.action_message,
Snackbar.LENGTH_INDEFINITE)
        .setBackgroundTint(getColor(R.color.green_dark))
        .setTextColor(getColor(R.color.snackbar_text))
        .setActionTextColor(getColor(R.color.amber_accent))
        .setAction(R.string.action_yes) {
            Toast.makeText(applicationContext, R.string.action_result,
Toast.LENGTH_SHORT).show()
        }
    showTracked(snackbar)
}

private fun showRoundedSnackbar(view: View) {
    val snackbar = Snackbar.make(view, R.string.rounded_message,
Snackbar.LENGTH_INDEFINITE)
        .setAction(R.string.dismiss_action) {
            updateEvent("закругленная панель закрыта")
        }
    SnackbarHelper.configure(this, snackbar)
    showTracked(snackbar)
}

private fun showTracked(snackbar: Snackbar) {
    activeSnackbar = snackbar
    snackbar.addCallback(object : Snackbar.Callback() {
        override fun onShown(shownSnackbar: Snackbar) {
            updateEvent("Snackbar показан")
        }

        override fun onDismissed(dismissedSnackbar: Snackbar, event:
Int) {
            updateEvent(dismissReason(event))
            if (activeSnackbar === dismissedSnackbar) {
                activeSnackbar = null
            }
        }
    })
    snackbar.show()
}

private fun dismissReason(event: Int): String = when (event) {
    Snackbar.Callback.DISMISS_EVENT_SWIPE -> "закрыт смахиванием"
    Snackbar.Callback.DISMISS_EVENT_ACTION -> "закрыт действием"
    Snackbar.Callback.DISMISS_EVENT_TIMEOUT -> "закрыт по таймауту-"
    Snackbar.Callback.DISMISS_EVENT_MANUAL -> "закрыт вручную"
    Snackbar.Callback.DISMISS_EVENT_CONSECUTIVE -> "заменен новым
Snackbar"
    else -> "закрыт"
```

```
    }

    private fun updateEvent(event: String) {
        Log.i(TAG, event)
        eventText.text = getString(R.string.event_format, event)
    }

    private companion object {
        const val TAG = "Lab6Snackbar"
    }
}
```

Kotlin. SnackbarHelper.kt

```
package ru.nstu.lab6.kotlinapp

import android.content.Context
import android.view.ViewGroup
import androidx.core.view.ViewCompat
import com.google.android.material.snackbar.Snackbar

object SnackbarHelper {
    fun configure(context: Context, snackbar: Snackbar) {
        val snackbarView = snackbar.view
        val layoutParams = snackbarView.layoutParams
        if (layoutParams is ViewGroup.MarginLayoutParams) {
            val margin = context.resources.getDimensionPixelSize(R.dimen.
   snackbar_margin)
            layoutParams.setMargins(margin, margin, margin, margin)
            snackbarView.layoutParams = layoutParams
        }
        snackbarView.setBackgroundResource(R.drawable.bg_snackbar_rounded)
        ViewCompat.setElevation(
            snackbarView,
            context.resources.getDimension(R.dimen.snackbar_elevation),
        )
        snackbar.setTextColor(context.getColor(R.color.snackbar_text))
        snackbar.setActionTextColor(context.getColor(R.color.amber_accent))
    }
}
```